

Creating Custom Methods for Sending E-Documents

An administrator can create various custom methods for sending e-documents or transaction response to different customers and vendors. You can use the custom sending methods to get status updates from a network.

To create a custom method for sending e-documents, the administrator must first create an e-document sending method plug-in implementation and then create an e-document sending method record for that implementation.

After custom sending methods are created, they become available for selection on e-document package records and transaction records.

See the following topics:

- [Creating a Custom Plug-in Implementation for Sending E-Documents](#)
- [Creating a Script for Sending E-Documents](#)
- [Creating an E-Document Sending Method Record](#)
- [Editing an E-Document Sending Method Record](#)

Creating a Custom Plug-in Implementation for Sending E-Documents

A custom plug-in implementation for sending e-documents or transaction response must be created so that it will be available for selection on the sending method record.

Create a JavaScript file for the custom plug-in implementation. The JavaScript file must be compatible with SuiteScript 2.0.

Note

This sample script uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript Debugger](#).

The following code is a sample custom plug-in implementation for sending e-documents.


```

/**
 * send - This function is the entry point of our plugin script
 * @param {Object} pluginContext
 * @param {String} pluginContext.scriptId
 * @param {String} pluginContext.sendMethodId
 * @param {String} pluginContext.eInvoiceContent
 * @param {Array} pluginContext.attachmentFileIds
 * @param {String} pluginContext.customPluginImpId
 * @param {Number} pluginContext.batchOwner
 * @param {Object} pluginContext.customer
 * @param {String} pluginContext.customer.id
 * @param {Array} pluginContext.customer.recipients
 * @param {Object} pluginContext.transaction
 * @param {String} pluginContext.transaction.number
 * @param {String} pluginContext.transaction.id
 * @param {String} pluginContext.transaction.poNum
 * @param {String} pluginContext.transaction.tranType
 * @param {Number} pluginContext.transaction.subsidiary
 * @param {Object} pluginContext.sender
 * @param {String} pluginContext.sender.id
 * @param {String} pluginContext.sender.name
 * @param {String} pluginContext.sender.email
 * @param {Number} pluginContext.userId
 *
 * @returns {Object} result
 * @returns {Boolean} result.success
 * @returns {String} result.message
 */
function send(pluginContext) {

    var MSG_NO_EMAIL = translator.getString("ei.sending.sendern
    var MSG_SENT_DETAILS = translator.getString("ei.sending.sen

    var senderDetails = pluginContext.sender;
    var customer = pluginContext.customer;
    var transaction = pluginContext.transaction;
    var recipientList = customer.recipients;
    var result = {};
    var parameters;

```

```

if (!senderDetails.email) {
    parameters = {
        EMPLOYEE_NAME: senderDetails.name
    };
    stringFormatter.setString(MSG_NO_EMAIL);
    stringFormatter.replaceParameters(parameters);
    result = {
        success: false,
        message: stringFormatter.toString()
    };
} else {
    var invoiceSendDetails = {
        number: transaction.number,
        poNumber: transaction.poNum,
        transactionType : transaction.type,
        eInvoiceContent: pluginContext.eInvoiceContent,
        attachmentFileIds: pluginContext.attachmentFileIds
    };
    notifier.notifyRecipient(senderDetails.id, recipientList);

    parameters = {
        SENDER: senderDetails.email,
        RECIPIENTS: recipientList.join(", ")
    };
    stringFormatter.setString(MSG_SENT_DETAILS);
    stringFormatter.replaceParameters(parameters);

    result = {
        success: true,
        message: stringFormatter.toString()
    };
}

return result;
}

return {
    send: send
};

```


Important

The sending method custom plug-in script must have the `@NSScriptType plugintypeimpl`.

After creating the script for plug-in implementation, upload it to *Customization > Plug-ins > Plug-in Implementations > New*. The type of the custom plug-in implementation must be 'Sending Plugin'. For more information, see [Custom Plug-in Creation](#).

Creating a Script for Sending E-Documents

Important

Sending methods must be created as custom plug-in implementations instead of scripts. You must recreate existing sending method scripts as new custom plug-in implementations of the type 'Sending Plugin'. For more information, see [Creating a Custom Plug-in Implementation for Sending E-Documents](#). The system won't support sending method scripts in NetSuite 2019.2.

An e-document sending method script must be a JavaScript file that is compatible with SuiteScript 2.0.

The script must return an object with the following function:

send(scriptContext)

Description	Executed when sending an e-document.
Returns	A result object.

Parameters

Note

The `scriptContext` and `result` parameters are JavaScript objects.

Parameter	Type	Required / Optional	Description
<code>scriptContext.scriptId</code>	string	required	The ID of the document in the file cabinet
<code>scriptContext.sendMethodId</code>	string	required	The ID of the customer's or vendor's selected sending method
<code>scriptContext.eInvoiceContent</code>	string	required	The e-document content as a string Note This content is the generated e-document.
<code>scriptContext.attachmentFileIds</code>	array of strings	optional	The internal ID of the generated e document pdf file in file cabinet
<code>scriptContext.customPluginImpId</code>	string	required	The id (Field ID: scriptid) of the custom plug-in implementation of type sending plug-in set for the transaction.
<code>scriptContext.transaction.id</code>	string	required	The ID of the e-document transaction Note The transaction id specified here is the ID of the document from which the e-document was generated.
<code>scriptContext.transaction.number</code>	string	required	The document number of the e-document transaction
<code>scriptContext.transaction.poNum</code>	string	optional	The PO/check number of the e-document transaction
<code>scriptContext.sender.id</code>	string	required	The ID of the designated sender of the e-document
<code>scriptContext.sender.name</code>	string	required	The name of the designated sender of the e-document
<code>scriptContext.sender.email</code>	string	required	The email address of the designated sender of the e-document
<code>scriptContext.userId</code>	number	required	This internal ID of the current logged in user, it can be used wherever there is a requirement to refer to the current logged in user. For example, to update e-document audit trail by shared module API, <code>userId</code> can be used in owner property.

 Note

This sample script uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see [SuiteScript Debugger](#).

The following code is a sample script for sending e-documents.

 COPY

```

/**
 * @NApiVersion 2.x
 * @NModuleScope Public
 */
define(["N/record"], function(record, error) {
    return {
        /**
         * send - Sample implementation: This will copy the e-document c
         Memo field
         *
         * @param {Object} plugInContext
         * @param {String} plugInContext.scriptId
         * @param {String} plugInContext.sendMethodId
         * @param {String} plugInContext.eInvoiceContent
         *
         * @param {Object} plugInContext.customer
         * @param {String} plugInContext.customer.id
         * @param {String[]} plugInContext.customer.recipients
         *
         * @param {Object} plugInContext.transaction
         * @param {String} plugInContext.transaction.id
         * @param {String} plugInContext.transaction.number
         * @param {String} plugInContext.transaction.poNum
         *
         * @param {Object} plugInContext.sender
         * @param {String} plugInContext.sender.id
         * @param {String} plugInContext.sender.name
         * @param {String} plugInContext.sender.email
         *
         *
         * @returns {Object} result
         * @returns {Boolean} result.success: determines
         * @returns {String} result.message: a failure message
         */
        send: function(plugInContext) {
            var result = {
                success: true,
                message: "Success"
            };

```

```

try {
    var rec = record.load({
        type: record.Type.INVOICE,
        id: plugInContext.transaction.id,
    });
    rec.setValue({
        fieldId: "memo",
        value: [
            "Script ID: " + plugInContext.scriptId,
            "Customer: " + plugInContext.customer.name,
            "Transaction: " + plugInContext.transaction.num
            "Sender: " + plugInContext.sender.name,
            "Recipients: " + plugInContext.customer.recipie
            "Content: " + plugInContext.eInvoiceContent].jo
        });
    rec.save();
} catch (e) {
    result.success = false;
    result.message = "Failure";
}
return result;
}
};
});

```

The sending method gets network status updates and returns document's unique identifier as soon as the document is being sent to the network. The method must also return an additional key `networkStatus` in the send method implementation response.

 COPY

```
networkStatus:{
    referenceId:"e34fedf343ferterjty",
    name:"SAT",
    status: "Certification Processed",
    updateDateTime: new Date('30/11/2022 3:32:14 PM')
}
```

These are the details of parameters added in the response:

Parameter	Type	Description
networkStatus.referenceId	String	It is a unique number generated by network API while certifying or sending e-document.
networkStatus.name	String	It is the name of a network. For instance, PEPPOL, SAT, ARIBA.
networkStatus.status	String	It displays a keyword representing one of the possible network statuses used by the network.
networkStatus.updateDateTime	String	It displays the time stamp (date and time) of the updated status.

The following code is a sample script of a sending method which returns a network status object.

 COPY

```
{
    success: true,
    message: "Sent E-Document successfully",
    networkStatus: {
        referenceId: "p1335ead",
        name: "PEPPOL",
        status: "Processed",
        updateDateTime: new Date('30/11/2022 3:32:14 PM')
    },
};
```

The script returns an object with the function `getStatus` when the sending method is used to get status updates from an API.

getStatus(scriptContext)

Description	Executed to get the network status updates.
Returns	A result object.

Parameters

Parameter	Type	Required / Optional	Description
scriptContext.transaction.id	string	required	The ID of the e-document transaction <i>Note</i> The transaction id specified here is the ID of the document from which the e-document was generated.
scriptContext.transaction.type	string	required	The type of the e-document transaction.

getStatus method: Response Parameters

Parameter	Type	Required/Optional	Description
success	boolean	required	In case of successful communication with network status check API, it is set as true. In all failed communication cases, it is set as false.
message	string	required	Message displayed in the E-document Audit Trail .
networkStatus.status	string	required if success parameter is true	Keyword representing one of the possible network statuses used by the network.
networkStatus.updateDateTime	string	required if success parameter is true	Time stamp (date and time) for the status

The following code is a sample script of returned response for getStatus function.

 COPY

```
{
    success: true,
    message: 'Get the latest network status of e-document is s
    networkStatus:{
        status: "Processed",
        updateDateTime: new Date('30/11/2022 3:32:14 PM')
    },
}
```

Important

To prevent permission errors, make sure the sending method script has the **@NModuleScope Public** JSDoc .

Creating an E-Document Sending Method Record

Make sure that you have created a sending method custom plug-in implementation before you create e-document sending method records. For more information, see [Creating a Custom Plug-in Implementation for Sending E-Documents](#).

To create an e-document sending method record:

1. Go to Setup > E-Documents > E-Document Sending Methods > New.
2. In the **Name** field, enter a name for the e-document sending method.
3. In the **E-Document Package** field, select the e-document package you want to associate this sending method with. For more information, see [Creating E-Document Packages](#).
4. In the **E-Document Sending Method Plugin Implementation** field, select the e-document plug-in implementation for this method.
5. In the **Sending Channel** field, enter the sending channel to use for this method. For example, **email**, **SOAP** or **REST**.

Note

If the sending channel is **email** (case-sensitive), the system validates the email recipients upon saving the customer or vendor record and when sending the e-document.

6. In the **Transaction Type** field, select one or more transaction types for which this sending method will be used. To select multiple transaction types, press and hold the **Ctrl** key while selecting the transaction types.

The Transaction Type field only displays the transaction types applicable to or supported by outbound e-document sending, which include:

- Cash Refund
- Cash Sale
- Credit Memo
- Customer Payment

- Customer Deposit
- Estimate
- Invoice
- Item Fulfillment
- Purchase Order
- Return Authorization
- Registered Custom Transaction Types
- Transfer Order
- Vendor Credit or Bill Credit

For more information, see [Transactions and Processes Supported by the Electronic Invoicing SuiteApp](#).

 Note

The selected transaction types cannot be modified after the sending method has been used in a transaction. You must remove the e-document sending method from the transaction before you can modify this field.

7. In the **Subsidiary** field, select the subsidiaries that this sending method will be associated with. To select multiple subsidiaries, press and hold the **Ctrl** key while selecting the subsidiaries.

If only this sending method is associated with a subsidiary, the supported transactions of that subsidiary will display this sending method on the E-Document Template field on the E-Document subtab. For more information, see [Multi-subsidiary Support in the Outbound Process](#).

8. (Optional) If the sending method is to be used for e-document certification, check the **Sending Method for Certification** box. For more information, see [E-Document Certification in the Outbound Process](#).

 Note

Only one certification sending method must be assigned to a combination of subsidiaries and transactions you select.

9. Click **Save**.

This sending method can now be selected on e-document package records.

If the **Inactive** box is checked, this record won't be available for selection.

Editing an E-Document Sending Method Record

To edit an e-document sending method record, go to Setup > E-Documents > E-Document Sending Methods. Open the e-document sending method in edit mode and modify information as needed, then click **Save**.

Sending method records must reference sending method plug-ins, instead of scripts, from the **E-Document Sending Method Plugin Implementation** field. Existing sending method scripts must be recreated as new custom plug-in implementations of the type 'Sending Plugin'. For more information, see [Creating a Custom Plug-in Implementation for Sending E-Documents](#).

Important

The system won't support sending method scripts in NetSuite 2019.2, but until that time, you can still edit and use existing sending method scripts.

Note

The **Transaction Type** field in the e-document sending method is disabled if the e-document sending method has been assigned to one or more transaction records. To enable the field, you must remove the e-document sending method from all transactions.

Related Topics

[Creating E-Document Sending Methods](#)
[Setting Up an Email Sending Method for E-Documents](#)

[General Notices](#)